

Comparison :

1. Model Size Comparison

Framework	Model Size	Compression Ratio
TensorFlow/Keras	286.33 KB	1.0x baseline
TensorFlow Lite	88.86 KB	3.22x
TF Lite Quantized	25.94 KB	11.04x

2. Accuracy Comparison

Framework	Test Accuracy	Accuracy Loss
TensorFlow/Keras	97.80%	baseline
TensorFlow Lite	97.80%	+0.00%
TF Lite Quantized	97.81%	+0.01%
TF Lite Micro	97.81%	+0.01%

3. Memory Usage Analysis

Framework	RAM Usage	Flash Usage	Total Footprint
TensorFlow/Keras	~200.0 MB	0.3 MB	~200.3 MB
TensorFlow Lite	~65.0 KB	88.9 KB	~153.9 KB
TF Lite Micro	9.0 KB	26.6 KB	35.6 KB

4. Inference Performance

Due to the absence of a physical microcontroller or designated edge hardware for this assignment, empirical inference performance metrics—such as actual inference time, energy consumption, and hardware-level throughput—could not be accurately measured. Consequently, quantitative data for this section has been omitted to maintain the empirical integrity of the report, as theoretical estimations may not accurately reflect real-world execution on bare-metal environments.

5. Development Complexity

TensorFlow Training (Python) requires 74 lines of code implementing a CNN model with Conv2D, MaxPooling, Flatten, and Dense layers. The implementation uses core dependencies including TensorFlow, Keras, and NumPy. Build complexity is minimal using pip install. Debugging utilizes Python's interactive environment and TensorFlow's

built-in training visualization.

TFLite Conversion and Quantization (Python) contains 173 lines of code implementing model conversion, full-integer quantization, and accuracy validation. Dependencies include tensorflow-lite and NumPy for quantization calibration. The build process requires careful configuration of converter settings for int8 quantization. Debugging involves manual quantization parameter verification and accuracy testing, with representative dataset calibration adding moderate complexity.

TensorFlow Lite Micro Deployment (C++) consists of 116 lines in `model_inference.cc` plus 2,217 lines in the auto-generated `model_data.cc`. This represents significant complexity increase from Python to C++. The implementation uses `tflite::MicroMutableOpResolver` with 8 specifically registered operators (Conv2D, MaxPool2D, FullyConnected, Reshape, Softmax, Shape, StridedSlice, Pack) rather than `AllOpsResolver`. Dependencies include TensorFlow Lite Micro, flatbuffers, gemmlowp, and ruy libraries. Build complexity is high with a Makefile requiring precise path configuration for multiple third-party libraries. The choice of minimal operator resolver reduces binary size but requires careful operator identification. Debugging in C++ uses `MicroPrintf` for embedded-style logging and requires manual tensor arena size tuning (optimized to 8,976 bytes through experimentation).

Overall Complexity Progression: Transition complexity escalates significantly from Python's high-level abstractions to C++'s low-level memory management. Python implementations benefit from dynamic typing and automated dependency resolution, while C++ demands explicit header inclusion and cross-compilation awareness. The operator resolver decision exemplifies embedded deployment trade-offs: minimal resolvers reduce memory footprint at the cost of requiring explicit operator registration and potential compatibility issues with model changes. Each part demonstrates appropriate complexity for its target platform, with Part 3 specifically addressing microcontroller constraints through memory optimization and minimal dependency inclusion.

Analysis:

1. Trade-off Analysis

The project demonstrates a clear trade-off progression across three deployment frameworks, each optimizing different aspects of the expressivity-efficiency-deployment triad.

TensorFlow/Keras prioritizes model expressivity with full access to Keras' comprehensive layer library and training capabilities. This framework achieves maximum flexibility in model architecture design at the cost of computational efficiency, resulting in larger model sizes and dependency-heavy deployment requirements. The trade-off favors rapid prototyping and experimentation over deployment optimization.

TensorFlow Lite shifts the balance toward computational efficiency and deployment feasibility. Through static graph optimization and full integer quantization, the framework reduces model size by over 90% (from 286.33 KB to 25.94 KB) while maintaining, and even slightly improving, accuracy (97.81% in Int8 vs. 97.80% in float32). The expressivity trade-off manifests in reduced operator support (TFLITE_BUILTINS_INT8 ops only) and quantization constraints that simplify model complexity for embedded compatibility.

TensorFlow Lite Micro represents the most deployment-constrained framework, sacrificing expressivity for extreme efficiency. The C++ implementation requires explicit operator registration (8 specific ops rather than comprehensive libraries) and manual memory management (8,976-byte tensor arena). Computational efficiency is maximized through static memory allocation and the absence of dynamic runtime dependencies, but model expressivity is limited to pre-determined operator sets compatible with microcontroller constraints. The framework enforces a "deployment-first" philosophy where model architecture must be designed within strict memory and computational boundaries from inception.

The progression illustrates a fundamental ML deployment principle: expressivity, efficiency, and deployment constraints exist in tension, with each framework representing a different equilibrium point along this three-dimensional trade-off surface.

2. Use Case Suitability

TensorFlow/Keras is chosen for cloud servers and high-performance hardware where

available computational resources are practically unlimited, easily accommodating footprints exceeding 200 MB. It operates without power constraints, relying on continuous power supplies. Latency requirements are typically managed by network bandwidth and server processing speed rather than on-device limitations. The model update frequency is extremely high, as cloud environments allow for immediate and seamless deployment of new weights. The required development team expertise focuses primarily on Python, rapid prototyping, and standard data science workflows.

TensorFlow Lite is selected for mobile devices and single-board computers that provide moderate computational resources. Power constraints are moderate, typically involving battery-powered devices that require efficient execution to preserve battery life. It meets strict low latency requirements by processing data on-device, entirely eliminating network transmission delays. The model update frequency is medium, as models are updated periodically via mobile application releases or over-the-air (OTA) updates. The development team requires expertise in translating Python models into mobile environments and managing cross-platform application integration.

TensorFlow Lite Micro is exclusively used for bare-metal microcontrollers where computational resources are extremely limited, bounded by strictly kilobyte-scale memory Flash footprints. Power constraints are severe, demanding models that can operate on coin-cell batteries or energy-harvesting systems for extended periods. It handles ultra-low latency requirements by running inference directly at the edge sensor level in real-time. The model update frequency is very low, as the model data array is often permanently flashed onto the device ROM and difficult to update post-deployment. The development team must possess in embedded C++, manual static memory allocation, Makefile construction, operator resolver configuration, and hardware-level debugging without standard stack traces.

3. Quantization Impact

The application of full integer quantization directly impacted the experimental results across four key metrics.

Regarding model size, quantization drastically reduced the storage footprint. By converting 32-bit float values to 8-bit integers, the model size decreased from 286.33 KB to 25.94 KB. This 11.04x compression ratio is the defining factor that makes deploying this model onto a memory-constrained microcontroller possible.

For model accuracy, the results showed an unexpected trend: the test set accuracy slightly increased from 97.80% in the baseline model to 97.81% in the INT8 model. This

result indicates that the loss of precision did not harm the model's predictive capability; instead, the lower precision likely provided a slight regularization effect by smoothing out the network weights, thereby reducing overfitting.

Regarding inference speed, although actual execution time was not measured on physical hardware, quantization theoretically accelerates inference. Processing 8-bit integer data requires significantly less computational overhead and memory bandwidth compared to 32-bit floating-point arithmetic, which leads to faster execution on edge devices.

Finally, quantization substantially increased development complexity. The quantization conversion process automatically injected implicit operators into the model structure. Consequently, deploying the model in C++ required manually debugging runtime memory allocation failures to identify and register these hidden operators. Furthermore, managing the quantized model required exact manual memory allocation, forcing a binary search testing process to lock down the strict minimum tensor arena size to exactly 8,976 bytes.

4. Deployment Considerations

In production, framework choice depends heavily on practical trade-offs rather than just technical metrics. Standard TensorFlow Lite requires expensive processors with operating system support, whereas TFLite Micro allows for mass deployment on extremely low-cost microcontrollers. Furthermore, maintenance mechanisms differ vastly; cloud models update seamlessly over the air, while embedded TFLite Micro models typically require risky, complete firmware flashes. Finally, team expertise is a crucial constraint, as TFLite Micro demands bare-metal C++ debugging skills that can present a significant hurdle for data science teams accustomed to Python-centric environments.

5. Future Directions

As edge hardware continues to diversify, framework specialization will inevitably shift from manual trial-and-error optimization toward highly automated hardware-software co-design. Future compilers will automatically profile target hardware, strip unnecessary code, and seamlessly generate the exact minimal resolvers and static memory allocations required. This evolution will drastically lower the barrier to entry for embedded machine learning, allowing developers to shift their focus purely back to model optimization and application design rather than low-level C++ debugging.